

ICLR 2022 | 离线强化学习，其实一个两层 MLP 就够了

UC Berkeley 与 CMU 把「监督学习做离线 RL」拆到最简：真正重要的只有模型容量与「条件变量」的选择

谈到离线强化学习（offline RL，只用一批预先收集好的数据、不与环境交互地学策略），大多数人的第一反应是价值函数：先学一个 Q 函数，再用它去改进策略。这类基于时序差分（TD, temporal-difference）的方法理论上很强，却出了名地难用，需要各种稳定化技巧和精细调参才能不发散。

于是有人换了个思路：把强化学习问题改写成一个「有条件的模仿学习」。核心洞察是，一条对某个目标来说很糟糕的轨迹，对另一个目标可能恰好是最优示范。只要给策略额外输入一个「想要达成的结果」，再用普通的监督学习去拟合数据里的动作，似乎就能避开价值函数。但这条路线一直很模糊：不同论文给出了互相矛盾的结论，有的说必须做 advantage weighting（优势加权），有的干脆搬出大型 Transformer 序列模型。到底哪些组件是必需的，哪些只是额外的复杂度？

这篇发表于 ICLR 2022 的论文（作者来自 UC Berkeley 与 Carnegie Mellon）不打算再提一个新算法，而是把这些方法统一到一个框架下，称作 RvS（Reinforcement Learning via Supervised Learning，用监督学习做强化学习），然后用大量实验把它拆解到最简。结论出人意料：一个只有两层全连接的前馈网络（MLP），单纯地最大化似然，就能追平那些依赖 TD 学习或 Transformer 的最新方法。真正决定成败的，只有两件事，模型容量和「条件变量」的选择。

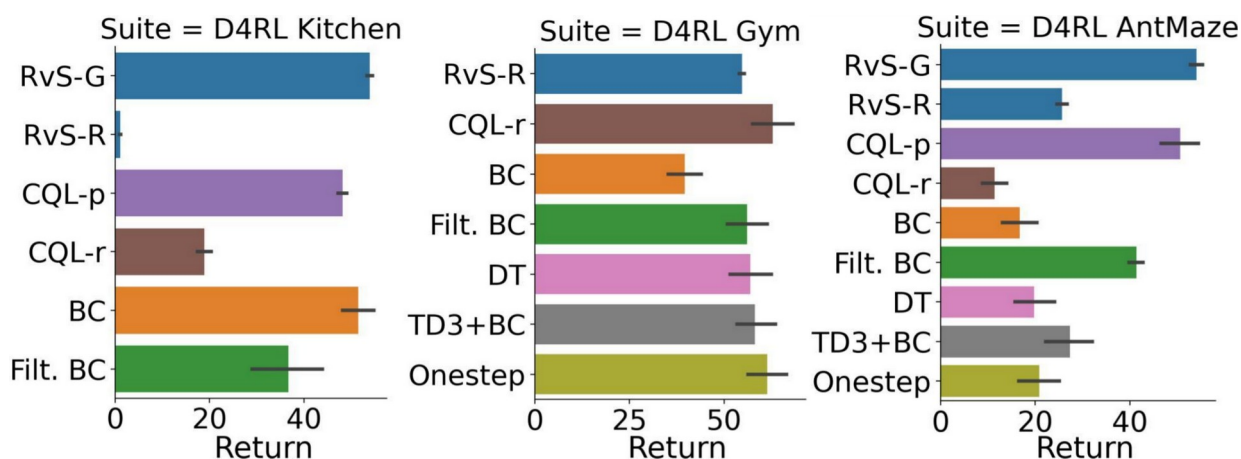


图 1 在 D4RL 的 Kitchen、Gym、AntMaze 三个套件上，条件于目标的 RvS-G（蓝）与条件于回报的 RvS-R（蓝）对比各类先前方法与基线。每根柱子是套件内多个任务的平均回报。仅凭一个前馈 MLP 的监督学习，RvS 就追平了使用 TD 学习和 Transformer 序列模型的方法。

论文链接：<https://arxiv.org/abs/2112.10751> 代码：<https://github.com/scottemmons/rvs>

为什么值得把方法「拆简」

基于价值的方法在离线和 off-policy 场景里长期占主导，也有漂亮的理论保证。但实际用起来门槛很高：要靠一堆技巧来稳定训练，还要小心调很多超参数。把 RL 转成有条件、过滤或加权的模仿学习，是一个诱人的替代方案，因为它只需要做监督学习。如果一个极简的监督学习配方真能追平这些复杂方法，那它对工程实践的意义就很直接：给从业者一份可靠的「操作手册」，同时也暴露出这类方法到底在哪里会失效。

论文的三点贡献可以这样概括。第一，它不发明新算法，而是把已有的目标条件（goal-conditioned）和回报条件（reward-conditioned）方法收进 RvS 这一个统一框架。第二，通过大规模实验把这些方法「蒸馏」到本质，证明一个两层 MLP 的最大似然训练就足以与最新方法竞争。第三，它明确指出哪些选择真正重要（模型容量、正则化、条件变量），并诚实地划出边界：在纯随机数据上，RvS 的表现明显弱于价值型方法，作者把它列为一个有待解决的开放问题。

RvS：一个刻意简单的框架

RvS 的设定很朴素。智能体处在一个马尔可夫决策过程中，我们训练一个「以结果为条件」的策略 $\pi(a | s, \omega)$ ，其中结果 ω 可以是一个未来要到达的目标状态（对应 RvS-G），也可以是未来若干步的平均回报（对应 RvS-R）。给定一批离线轨迹，RvS 用「事后重标注」（hindsight relabeling）来构造训练集：轨迹里实际发生过的每个动作，都被当成「达成它后来真正达成的那个结果」的示范。

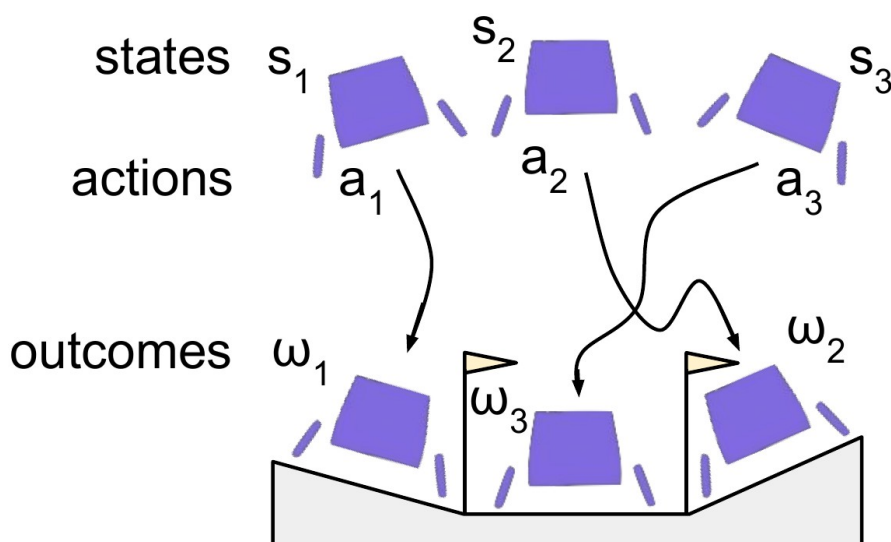


图 2 事后重标注意：轨迹中的状态 s 与动作 a ，被重新标注为通往它们实际到达的结果 ω 的示范。这样一来，一批未经筛选的经验就变成了「状态+目标 $\rightarrow$ 动作」的监督数据。

有了这样的数据，训练目标就只是最大化在条件策略下观测动作的对数似然：把结果 ω 拼接到输入状态 s 上，让一个两层全连接 MLP 去预测动作即可。这里没有 advantage weighting，没

有 TD 自举，也没有 Transformer，只有一个在重标注经验上的最大似然目标。测试时，我们再给它指定想要的结果（一个目标状态或一个目标回报），让策略去执行。

在哪些任务上评测

评测覆盖面很广，横跨导航、运动控制和机械操作。在 D4RL 基准上用了三个套件：AntMaze（8 自由度四足机器人走迷宫）、Gym Locomotion（HalfCheetah、Hopper、Walker，数据质量从 random 到 medium-expert 不等）、以及 Franka Kitchen（9 自由度机械臂操作，由人类示范构成）。此外还用了 GCSL 套件里的目标条件任务，包括二维导航、Sawyer 机械臂、Lunar Lander 和机器爪；这些任务被改造成离线设置，数据由随机策略采集。所有分数都归一化到 [0, 100] 区间以便比较。

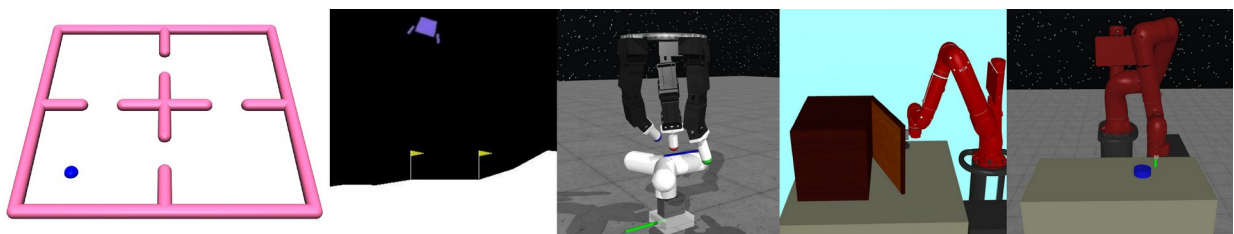


图 3 GCSL 套件中的几个目标条件环境（从左到右：FourRooms 导航、Lunar Lander、机器爪 Claw、开门 Door、推物 Pusher）。它们与 D4RL 的运动和操作任务一起，构成了从导航到精细操作的广谱评测。

结果：两层 MLP 就能达到当前最优

结论相当醒目。仅用一个两层前馈网络加最大似然训练，RvS 在多个套件上都达到了当前最优。在 AntMaze 上，目标条件的 RvS-G 平均得分 53.5，略微超过最强的价值型基线 CQL-p 的 50.6；在 Kitchen 上达到 54.0；在 GCSL 套件上得到 62.0，超过在线版 GCSL 的 58.0，而它用的还只是离线的随机策略数据。在 Gym Locomotion 上，回报条件的 RvS-R 追平了 Decision Transformer，却只用一个 MLP 而非大型 Transformer。

更反直觉的一点在于「子轨迹拼接」（subtrajectory stitching）。Kitchen 的 mixed 数据集和全部 AntMaze 任务，都需要把不同轨迹的片段拼起来才能完成，人们一直认为这类任务非得靠动态规划式的价值方法不可。但 RvS-G 在这里也能跟上，说明监督学习并不像想象中那样天然缺乏「拼接」能力。当然要如实说明：在 Gym 上价值型方法（如 CQL-r 的 63.0）仍然更高，RvS 的定位是「用最简手段追平」，而非全面碾压。

表 1 各套件归一化平均分（节选自论文表 1，分数区间 [0, 100]）。

套件 Suite	BC	Decision Transformer	最佳价值型基线	RvS-R	RvS-G
D4RL AntMaze	16.7	19.8	50.6 (CQL-p)	25.6	53.5
D4RL Gym	39.6	57.0	63.0 (CQL-r)	54.7	—
D4RL Kitchen	51.5	—	48.2 (CQL-p)	1.0	54.0
GCSL	—	—	58.0 (在线 GCSL)	—	62.0

注：GCSL 一栏的 58.0 为在线 GCSL 方法。带 * 的部分基线在论文中使用 AntMaze v0 版本，细节见原文附录 A。RvS-G 未在以回报最大化表述的 Gym 上运行，故留空。

真正重要的两件事：容量与正则化

既然模型这么简单，成败就落在少数几个设计选择上。论文的消融实验把它们一一钉死。第一是容量：表现最好的网络比标准在线 RL 或模仿学习里用的要大，把隐藏层宽度一路加到 1024 通常都还在涨。第二是正则化：dropout 并非总是好的。它在小规模、由人类示范构成的 kitchen-complete 上能提升表现，却在 hopper-medium-expert 和 antmaze-medium-play 上反而有害。

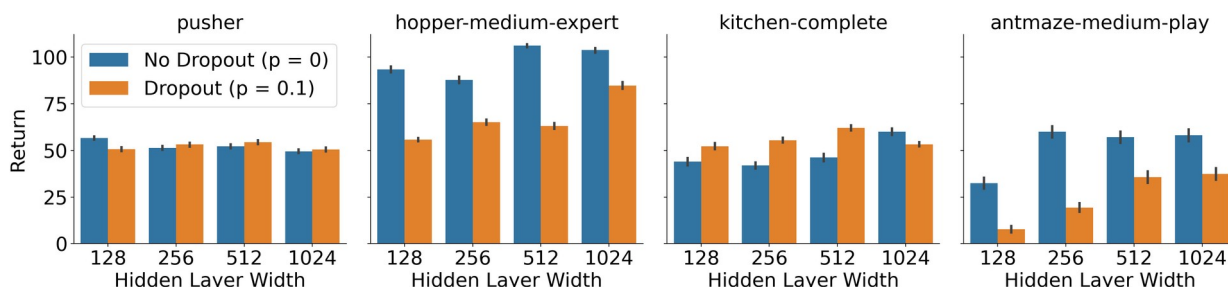


图 4 容量与正则化的消融：在四个环境上改变网络宽度（横轴）与是否使用 dropout（蓝为无、橙为 $p=0.1$ ）。更大的网络在 hopper-medium-expert 和 kitchen-complete 上更好，印证了高容量策略网络的重要性；而 dropout 只在 kitchen-complete 这类小数据集上稳定有益。

一个自然的解释是：kitchen-complete 数据量小、又来自人类示范，更容易过拟合，所以需要 dropout 来正则化；而在数据更充足或本就偏随机的任务上，强行加 dropout 反而限制了策略容量。换句话说，高容量的策略网络与「恰到好处」的正则化要配合使用，才能拿到好结果。

第三个选择是策略的输出分布。论文比较了在离散化动作空间上的类别分布（categorical）与连续动作空间上的单峰高斯（Gaussian）。在 GCSL 套件里，类别分布总体上不逊于甚至优于单峰高斯，这又一次呼应了「更高的策略容量更有帮助」这一主线。此外还有一个提醒调参者的观察：验证损失与最终表现只是松散相关，因此它并不是一个可靠的调参信号。

把这三点合起来看，它们其实指向同一个方向：RvS 的成败主要取决于策略网络的表达能力，而不是某个花哨的训练技巧。容量要够大、正则化要因数据而异、输出分布要足够灵活，本质上都是在给这个简单的 MLP 更多拟合数据的自由度。这也解释了为什么不能只盯着验证损失去调参，论文观察到几组超参数设置的验证损失几乎相同，最终的评测回报却可能相差明显，说明真正起作用的因素并没有被验证损失完全捕捉到。

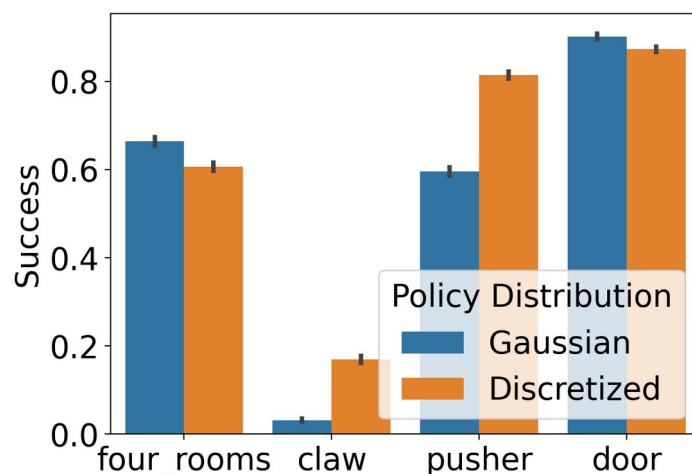


图 5 在 GCSL 套件四个环境上，离散化动作空间的类别分布（橙）总体上匹配或超过连续动作空间的单峰高斯（蓝）。这与「更高容量的 RvS 模型表现更好」的整体规律一致。

一份可直接上手的操作手册

把这些拼起来，论文其实交出了一份从业者的「操作手册」：做离线 RL 的 supervised 版本，你不需要 advantage weighting，也不需要 Transformer。一个两层前馈网络、单纯最大化似然，就能与当前最优竞争，前提是把两件事做对：一是仔细调好模型的容量与正则化（先把网络加宽到性能饱和，再视数据规模酌情加一点 dropout）；二是选对「条件变量」，也就是根据任务在目标（goal）与回报（reward）之间做取舍。

论文也没有回避它的边界。在纯随机数据上，TD 学习仍然占优，RvS 明显更弱，作者把这明确列为一个开放问题。对读者而言，这个结论的价值恰恰在于它的克制：它没有宣称 supervised 「解决」了 offline RL，而是清楚地告诉你，在什么条件下最简单的方法就已经足够好，以及它会在哪里失手。